

Московский государственный университет имени М.В. Ломоносова
Физический факультет
Кафедра общей физики и волновых процессов

Отчет по практическому заданию №1
Основы математического моделирования

Выполнил студент 325 группы
Делекторский Никита Юрьевич

Москва
2023

I. Постановка задачи (задача № 10)

Используя схему бегущего счета и итерационные методы, решить задачу:

$$\begin{cases} \frac{\partial u}{\partial t} - \arctg(u) \frac{\partial u}{\partial x} = 0, & -1 \leq x < 0 \\ u(x, 0) = -\sin(\pi x) \\ u(0, t) = 0 \end{cases} \quad (1)$$

II. Характеристики уравнения

Исследуем существование однозначного решения в данной области определения, т.е. определим, претерпевает ли решение разрыв в области $D: \{x \in [-1, 0], t \in (0, T)\}$, где T – любое. Если точки пересечения есть, то это означает, что в этих точках существует столько решений, сколько характеристик в них пересекаются. Характеристика $x(t)$ на которой $u = \text{const}$:

$$\frac{dt}{1} = \frac{dx}{-\arctg(u)}, \quad (2)$$

то есть характеристики – прямые линии с наклоном:

$$\frac{dx}{dt} = -\arctg(u), \quad (3)$$

который определяется начальными либо граничными условиями в зависимости от того, через какой из слоёв $t = 0$ или $x = 0$ проходит характеристика. В случае прохождения характеристики через границу $x = 0$ она является прямой, направленной параллельно оси времени и не пересекающей характеристики, идущие из начальных условий. Построим характеристики при изменении координаты x от -1 до 0 и времени t от 0 до 5 .

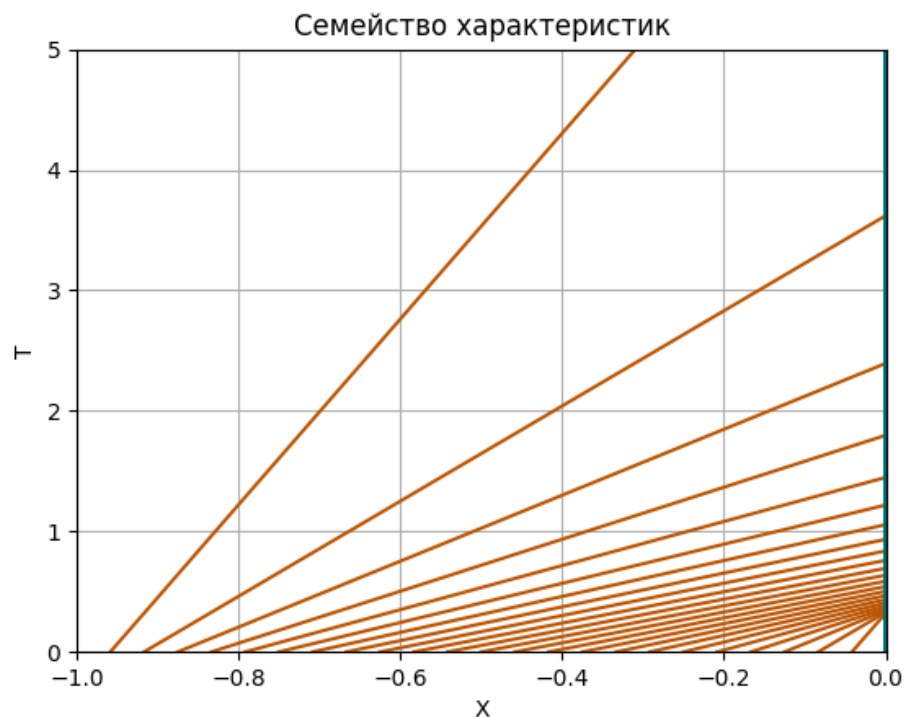


Рис. 1. Семейство характеристик для различных x_0 и t_0

На графике, представленном на Рис. 1, видно, что никакие характеристики не пересекаются при $t > 0$ и $-1 \leq x < 0$, из чего следует, что решение в таких пределах изменения 1 координаты и времени определено однозначно.

III. Метод решения и разностная аппроксимация

Введём разностную сетку:

$$\omega: \left\{ x_i = ih, i = 0, \dots, N, h = -\frac{1}{N}; t_j = j\tau, j = 0, \dots, M, \tau = \frac{T}{M} \right\}, \quad (4)$$

где τ – шаг по времени, h – шаг по x , N – число узлов вдоль оси x , M – количество узлов по времени. Для начала приведём исходное уравнение к дивергентному виду, выделим производную сложной функции:

$$\frac{\partial u}{\partial t} - \frac{\partial \left(u \cdot \arctan(u) - \frac{\ln \sqrt{1+u^2}}{2} \right)}{\partial x} = 0 \quad (5)$$

Введём функцию $u_i^j = u(x_i, t_j)$. Воспользуемся шаблоном вида

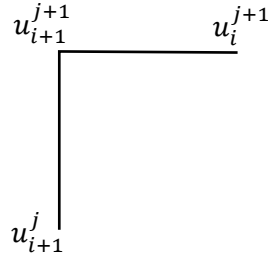


Рис. 2. Шаблон схемы бегущего счёта

Это неявная и абсолютно устойчивая схема. В левой верхней точке ищется значение сеточной функции при известных значениях в правой и нижней точках. Разностная аппроксимация задачи принимает вид:

$$\left\{ \begin{array}{l} \frac{u_{i+1}^{j+1} - u_{i+1}^j}{\tau} - \frac{\left[\left(u_{i+1}^{j+1} \arctan u_{i+1}^{j+1} - \frac{\ln(1 + (u_{i+1}^{j+1})^2)}{2} \right) - \left(u_i^{j+1} \arctan u_i^{j+1} - \frac{\ln(1 + (u_i^{j+1})^2)}{2} \right) \right]}{h} = 0 \\ u_i^0 = -\sin \pi x_i \\ u_0^j = 0 \end{array} \right. \quad (6)$$

Получаем нелинейное уравнение на u_{i+1}^{j+1} . Будем решать его методом Ньютона: если обозначить приближённое значение корня на текущей s -ой итерации как $\hat{u}_s$, то приближение корня на следующей итерации вычисляется как

$$\hat{u}^{s+1} = \hat{u}^s - \frac{f(\hat{u}^s)}{f'(\hat{u}^s)}, \text{ где используется функция} \quad (7)$$

$$f(\hat{u}^s) = \frac{\hat{u}^s - u_{i+1}^j}{\tau} - \frac{\left[\left(\hat{u}^s \arctan \hat{u}^s - \frac{\ln(1 + (\hat{u}^s)^2)}{2} \right) - \left(u_i^{j+1} \arctan u_i^{j+1} - \frac{\ln(1 + (u_i^{j+1})^2)}{2} \right) \right]}{h} \quad (8)$$

и её производная

$$f'(\hat{u}^s) = \frac{1}{\tau} - \frac{\arctan \hat{u}^s}{h} \quad (9)$$

Вычисления продолжаются до достижения заранее заданной точности, то есть до момента, когда оказывается выполненным условие $|\hat{u}^{s+1} - \hat{u}^s| \leq \varepsilon$, где ε задаётся заранее. За начальное приближение к корню возьмём значение сеточной функции на прошлом временном слое и из того же координатного слоя.

IV. Численное решение

Ниже представлен код программы реализующий численное решение поставленной задачи. Программа написана на языке Python. Количество шагов по оси координат задано $Nx = 400$ (шаг $h = 0.0025$) по оси времени – $Nt = 1000$ (шаг $\tau = 0.005$).

```
import numpy as np
import matplotlib.pyplot as plt
from numpy import arctan, sin, pi, log, sqrt
from numba import njit
from tqdm import tqdm
import time
import imageio

# Задаем начальные условия
X_START, X_END = 0, -1          # Длина сетки
T_START, T_END = 0, 5          # Интервал времени
epsilon = 1e-8
Nx, Nt = 400, 1000
saving = True                   # Сохранение графиков и анимаций

def draw_characteristics():      # Рисуем характеристики
    for x_0 in np.linspace(0, -1, 25):
        x_1 = x_0 + arctan(-sin(pi * x_0)) * T
        plt.plot(x_1, T, color='#BA5400', label='$t_0 = 0$')

    for t_0 in range(5):
        x_2 = (T - t_0) * arctan(0)
        plt.plot(x_2, T, color='#007070', label='$x_0 = 0$')

    plt.ylim(T_START, T_END)
    plt.xlim(X_END, 0.001)
    plt.grid()
    plt.xlabel('X')
    plt.ylabel('T')
    plt.title('Семейство характеристик')
    plt.show()

X = np.linspace(X_END, X_START, Nx) # Разбивка рассматриваемой области по координате x
T = np.linspace(T_START, T_END, Nt) # Разбивка интервала времени по времени

@njit                               # Ускорение расчётов
def F(m, n, y):
    return arctan(y[m][n]) * y[m][n] - log(sqrt(1+y[m][n]*y[m][n]))

@njit
```

```

def der_f(m1, n1, y):
    return 1 / 2 / ht - arctan(y[m1][n1]) / 2 / hx

@njit
def f(mp1, np1, y):
    n = np1 - 1
    m = mp1 - 1
    return (y[mp1][n] - y[m][n] + y[mp1][np1] - y[m][np1]) / (2 * ht) - (
        F(mp1, np1, y) - F(mp1, n, y) + F(m, np1, y) - F(m, n, y)) / (2 * hx)

# Параметры сетки
hx = (X_END - X_START) / (Nx - 1)
ht = (T_END - T_START) / (Nt - 1)
y = np.zeros((Nt, Nx))

timer = time.time()

# Начальное условие
y[0, :] = -sin(X * pi) # initial

# ----- СЧИТАЕМ ПО ЧЕТЫРЁХТОЧЕЧНОЙ СХЕМЕ -----

for m in tqdm(np.arange(Nt)[0:Nt - 1], desc='Расчёт по четырёхточечной схеме'):
    for n in np.arange(Nx)[0:Nx - 1]:
        eps = epsilon + 1
        while eps > epsilon:
            ep = f(m + 1, n + 1, y) / der_f(m + 1, n + 1, y)
            y[m + 1][n + 1] = y[m + 1][n + 1] - ep
            eps = np.abs(ep)

print(f'Четырёхточечная схема посчитана за {time.time() - timer} секунд')

xn = np.linspace(X_START, X_END, Nx)
tm = np.linspace(T_START, T_END, Nt)

x_grid, t_grid = np.meshgrid(xn, tm)
plt.ion()
if saving:
    step = 1
else:
    step = 10
fig = plt.figure(figsize=(8, 6))
for timer in tqdm(range(0, 360, step), desc='Анимация расчёта по четырёхточечной схеме'):
    plt.clf()
    ax_3d = plt.subplot(projection='3d')
    ax_3d.plot_surface(x_grid, t_grid, y, cmap='plasma')
    ax_3d.set_xlabel('X')
    ax_3d.set_ylabel('T')
    ax_3d.set_zlabel('U')
    # ax_3d.set_zlim(-30, 30)
    ax_3d.view_init(elev=0 + timer / 15, azim=timer)
    ax_3d.dist = 8
    plt.title('Численное решение уравнения переноса\nЧетырёхточечная схема')
    plt.draw()
    plt.gcf().canvas.flush_events()
    if saving:
        plt.savefig(f'Четырёхточечная схема/{timer}.png')
# plt.ioff()
y = np.transpose(y)

# ----- СЧИТАЕМ ПО ТРЁХТОЧЕЧНОЙ СХЕМЕ -----

x = np.linspace(X_END, X_START, Nx)
t = np.linspace(T_START, T_END, Nt)

u_2 = np.zeros((Nx, Nt)) # Пустой массив для проверки
u_2[:, 0] = -sin(x * pi)

@njit
def p(v):
    return -arctan(v) * v + log(sqrt(1 + v * v))

```

```

@njit
def f_2(v, c1, c2):
    return (v - c1) / tau + (p(c2) - p(v)) / h

@njit
def df(v):
    return 1 / tau - -arctan(v) / h

@njit
def solve(c1, c2):
    u1 = c2
    delta = eps + 1
    while delta > eps:
        u = u1
        u1 = u - f_2(u, c1, c2) / df(u)
        delta = abs(u1 - u)
    return u1

timer = time.time()

for i in tqdm(range(Nx - 2, -1, -1), desc='Расчёт по трёхточечной схеме'):
    for j in range(1, Nt):
        u_2[i, j] = solve(u_2[i, j - 1], u_2[i + 1, j])

u_2 = np.flip(u_2, axis=0)
print(f'Трёхточечная схема посчитана за {time.time()-timer} секунд')

x_grid, t_grid = np.meshgrid(xn, tm)
plt.ion()
fig = plt.figure(figsize=(8, 6))
if saving:
    step = 1
else:
    step = 10
for timer in tqdm(range(0, 360, step), desc='Анимация расчёта по трёхточечной схеме'):
    plt.clf()
    ax_3d = plt.subplot(projection='3d')
    ax_3d.plot_surface(x_grid, t_grid, u_2.T, cmap='plasma')
    ax_3d.set_xlabel('X')
    ax_3d.set_ylabel('T')
    ax_3d.set_zlabel('U')
    ax_3d.view_init(elev=0 + timer / 15, azimuth=timer)
    ax_3d.dist = 8
    plt.title('Численное решение уравнения переноса\nТрёхточечная схема')
    plt.draw()
    plt.gcf().canvas.flush_events()
    if saving:
        plt.savefig(f'Трёхточечная схема/{timer}.png')

x_grid, t_grid = np.meshgrid(tm, xn)
frames = []
plt.ion()
fig = plt.figure(figsize=(8, 6))
if saving:
    step = 1
else:
    step = 10
for timer in tqdm(range(0, 360, step), desc='Анимация относительной ошибки'):
    plt.clf()
    ax_3d = plt.subplot(projection='3d')
    ax_3d.plot_surface(x_grid, t_grid, y-u_2, cmap='plasma')
    ax_3d.set_xlabel('T')
    ax_3d.set_ylabel('X')
    ax_3d.set_zlabel('U')
    # ax_3d.set_zlim(-30, 30)
    ax_3d.view_init(elev=0 + timer / 15, azimuth=timer)
    ax_3d.dist = 8
    plt.title('Численное решение уравнения переноса\nОтносительная ошибка')
    plt.draw()
    plt.gcf().canvas.flush_events()
    if saving:
        plt.savefig(f'Относительная ошибка/{timer}.png')

print(f'\nМаксимальная ошибка составила {np.max(abs(y - u_2))}')

```

```

print(f'\nМинимальная ошибка составила {np.min(abs(y - u_2))}')
print(f'\nСредняя по модулю ошибка составила {np.mean(abs(y - u_2))}')

# ----- СОЗДАЁМ АНИМАЦИИ -----

if saving:
    frames = []
    for t in range(360):
        image = imageio.v2.imread(f'Четырёхточечная схема/{t}.png')
        frames.append(image)
    imageio.mimsave('Четырёхточечная схема.gif', # output gif
                    frames, # array of input frames
                    fps=30)

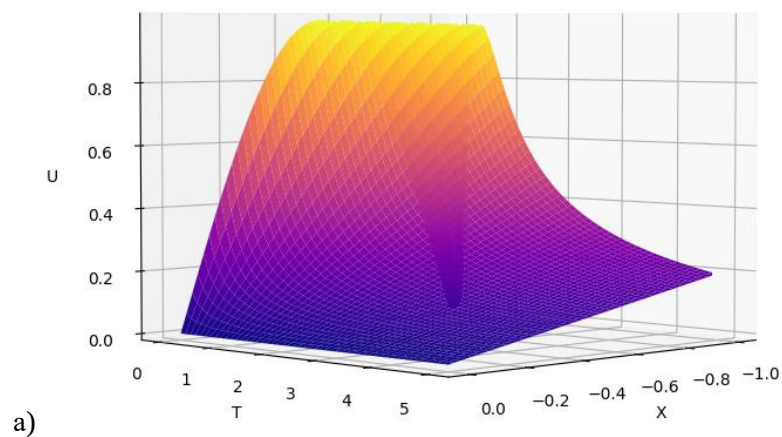
    frames = []
    for t in range(360):
        image = imageio.v2.imread(f'Трёхточечная схема/{t}.png')
        frames.append(image)
    imageio.mimsave('Трёхточечная схема.gif', # output gif
                    frames, # array of input frames
                    fps=30)

    frames = []
    for t in range(360):
        image = imageio.v2.imread(f'Относительная ошибка/{t}.png')
        frames.append(image)
    imageio.mimsave('Относительная ошибка.gif', # output gif
                    frames, # array of input frames
                    fps=30)

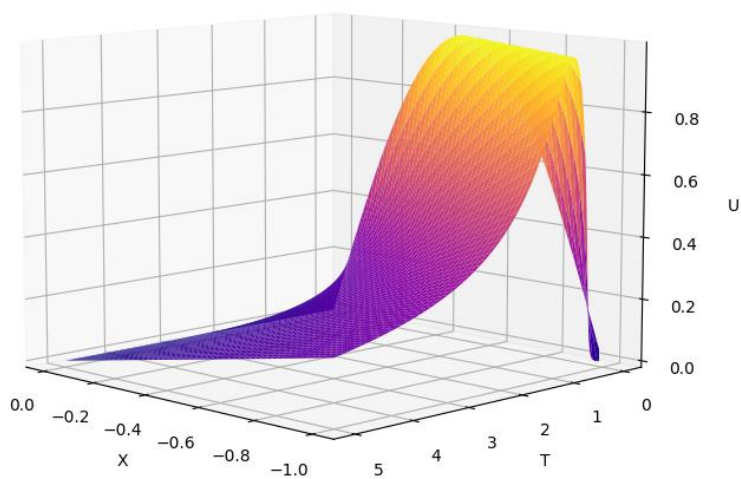
```

V. Результаты

Численное решение уравнения переноса
Трёхточечная схема

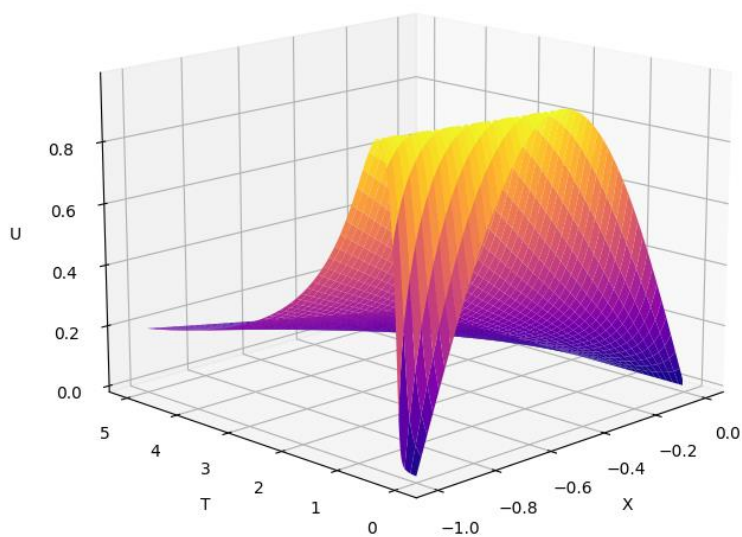


Численное решение уравнения переноса
Трёхточечная схема



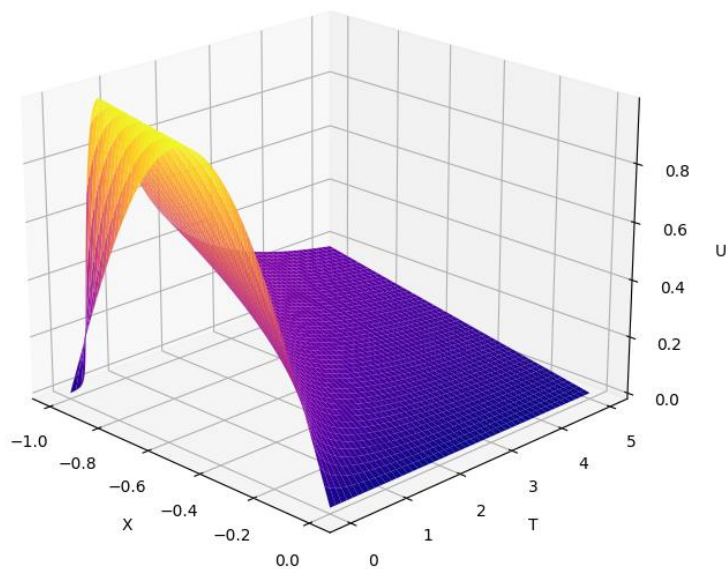
б)

Численное решение уравнения переноса
Трёхточечная схема



в)

Численное решение уравнения переноса
Трёхточечная схема



г)

Рис. 3. Результаты численного решения в трёхмерном виде

VI. Проверка результатов

Для проверки результатов достаточно убедиться, что другой шаблон даст такой же результат с маленьким отклонением на уровне погрешности. Используем четырёхточечный шаблон, представленный на рисунке 4.

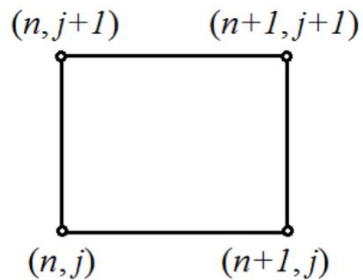
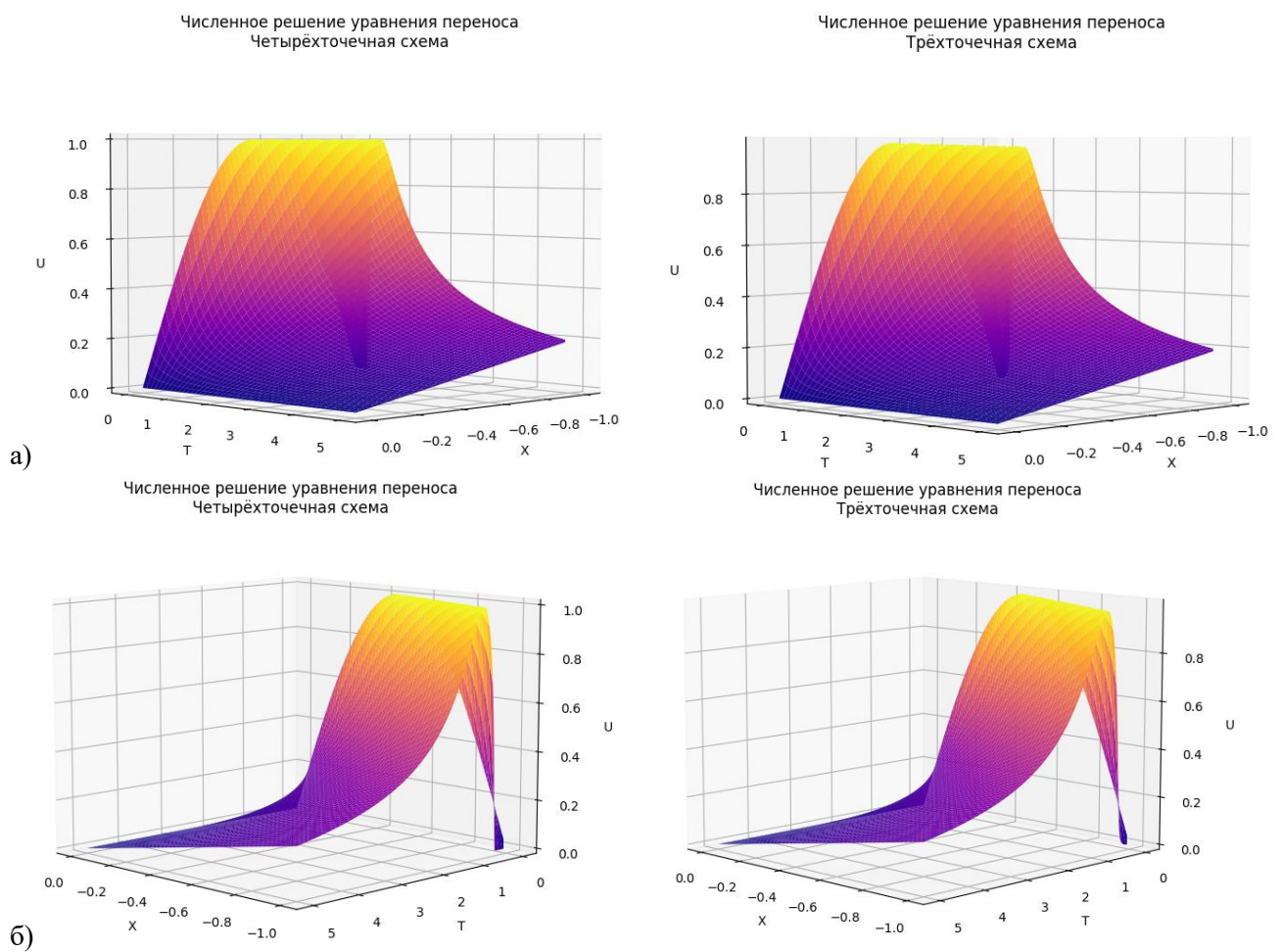


Рис. 4. Трёхточечный шаблон схемы бегущего счёта

Схема строится по аналогии с предыдущей. Сетка и все связанные с ней параметры должны быть такими же. На рисунке 5 представлены результаты численного моделирования.



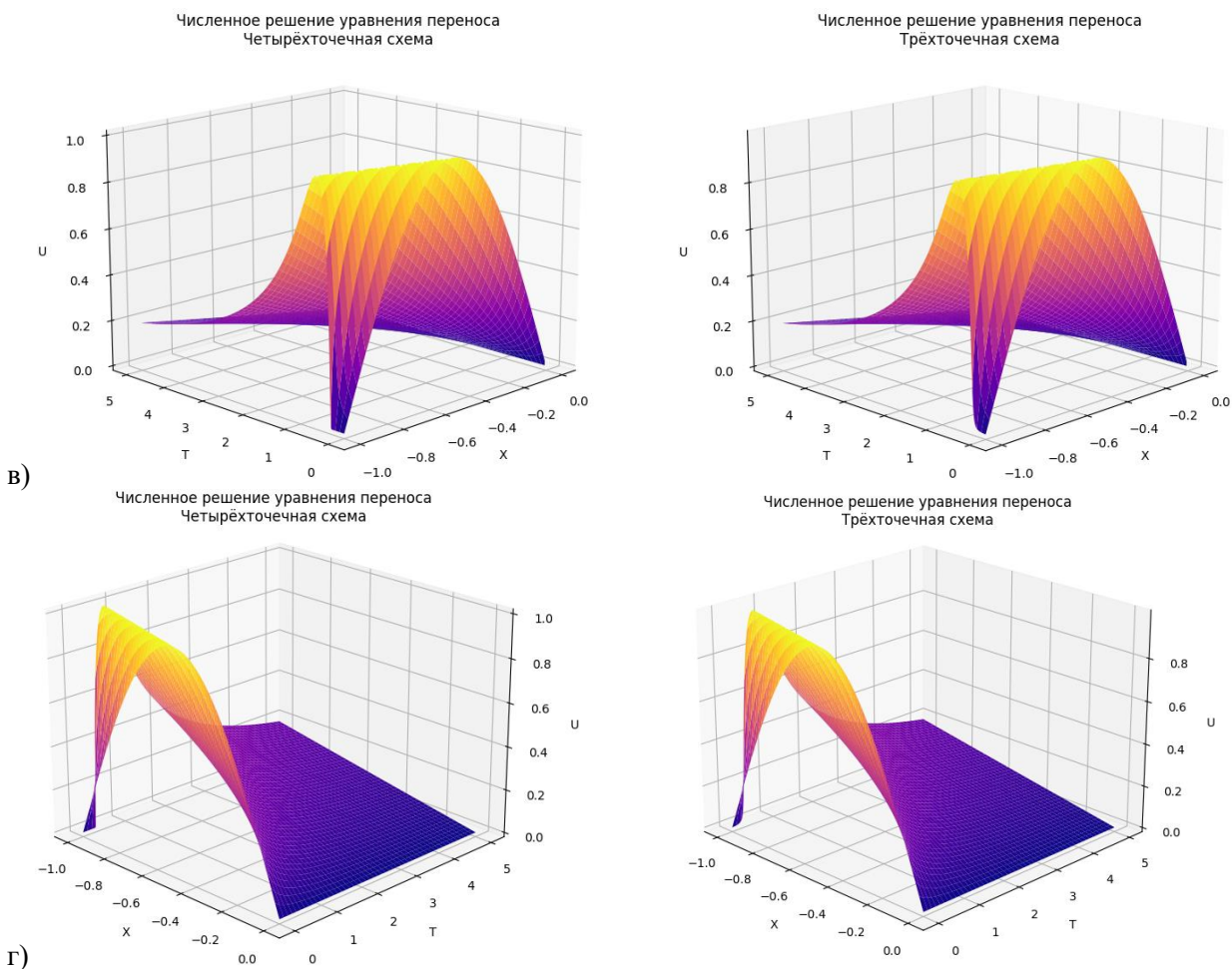
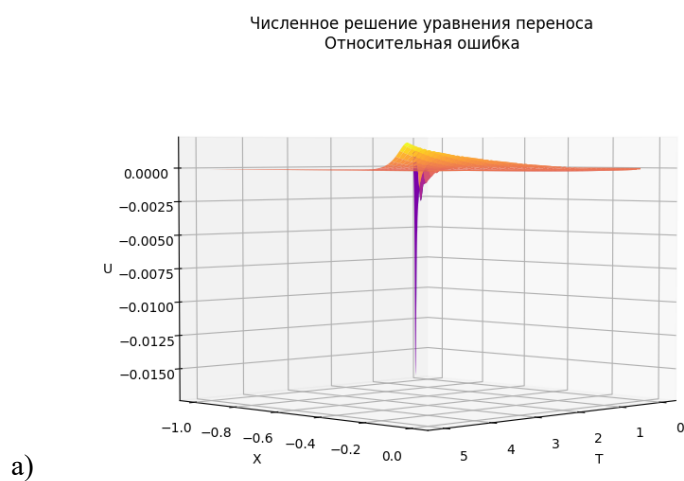
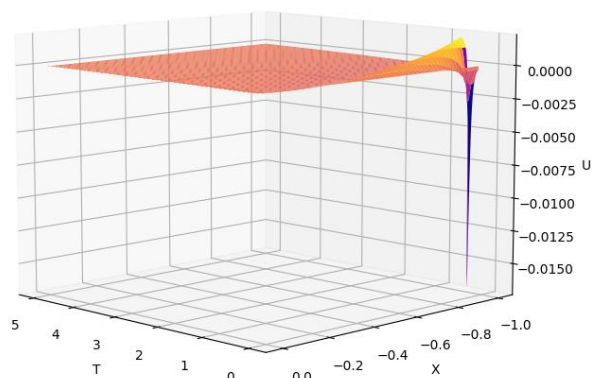


Рис. 5. Результаты численного моделирования для двух схем

Для большей наглядности разницы между результатами решений на рисунке 6 приведена их разность в трёхмерном виде. Средняя ошибка по модулю составила $\Delta U = 0.0017$.

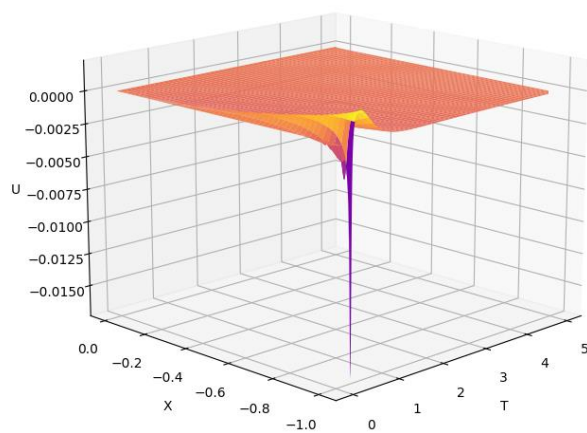


Численное решение уравнения переноса
Относительная ошибка



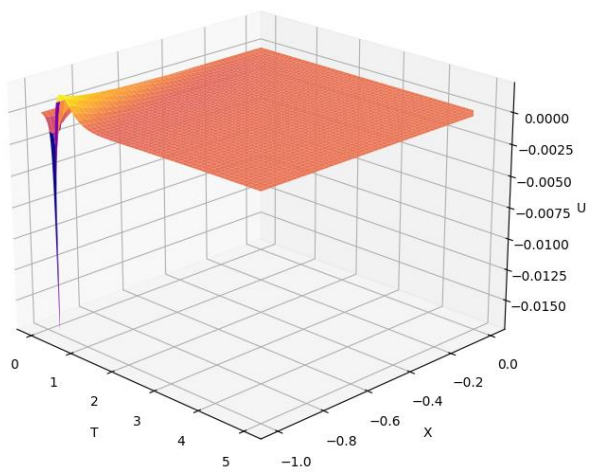
б)

Численное решение уравнения переноса
Относительная ошибка



в)

Численное решение уравнения переноса
Относительная ошибка



г)

Рис. 6. График относительной ошибки

VII. Заключение

В настоящей работе было проведено численное решение начально-краевой задачи для квазилинейного уравнения переноса с использованием схемы бегущего счета и итерационных методов (постановку задачи см. в пункте 1).

Для поиска решения в области $-1 \leq x < 0$ и $0 \leq t < T$ была введена равномерная прямоугольная сетка с шагом h по координате x и с шагом τ по времени t . Шаблон используемой схемы бегущего счета представлен на Рис. 2. Для решения трансцендентного уравнения относительно искомого значения сеточной функции на каждом шаге использовался итерационный метод Ньютона. Полное описание метода решения задачи представлено в пункте 3.

Программа была написана на языке Python без применения встроенных функций и библиотек, способных упростить решение задачи. Количество шагов по оси координат задано $Nx = 400$ (шаг $h = 0.0025$) по оси времени – $Nt = 1000$ (шаг $\tau = 0.005$). Полный код программы представлен в пункте 4.

По результатам сравнения решений двумя шаблонами схем бегущего счёта в пункте 6 было установлено, что среднее отклонение по модулю равно 0.0017, поэтому решение можно считать верным.